Measuring AI Value: A Structured-Use Pilot

Nov 11, 2026 · @Daryl Walleck

The story

The ask: move most of the team from the $40 Kiro plan to the $100 plan, and the heaviest users to the $200 plan. The pilot below is how we show each upgrade pays for itself.

How we use AI decides whether it is worth the spend, and we can measure the difference. Everyone already uses AI, so "AI vs. no AI" cannot be measured and is not the real question. We will compare structured AI use (the mattpocock/skills workflow) with ad hoc use in a bounded pilot, judged on criteria agreed before it starts.

The data needs to support four claims:

1. **Senior review time is protected.** Structured AI use reduces the senior review effort each PR needs. That is our scarcest resource and our biggest bottleneck.
2. **Delivery gets more predictable.** Cycle times become more consistent, so forecasts tighten and commitments to stakeholders become more reliable.
3. **Each tier upgrade pays for itself.** Spend is already set as flat-rate seats, so the question is outcomes per seat. The $100 Pro tier's newer models need to deliver enough beyond the $40 tier (capped at Sonnet 4.6 and Opus 4.5) to justify the $60 difference. The $200 tier goes to people who keep hitting the $100 tier's credit limit.
4. **Quality holds.** None of the gains come at the cost of defects, reverts or reopened work. Where results show no improvement, we report that too.

This argument is about output, quality and predictability, not hours saved. AI spend is added to payroll, not a replacement for it, so its value has to show up in what gets delivered.

How we measure fairly

Each person's work is compared against their own baseline, at the level of the work, with no individual ever reported.

- **Structured vs. ad hoc, per piece of work.** Each agent session is labeled with the skills it used and rolled up to its issue and PR. The same person's work is compared with and without the skills.

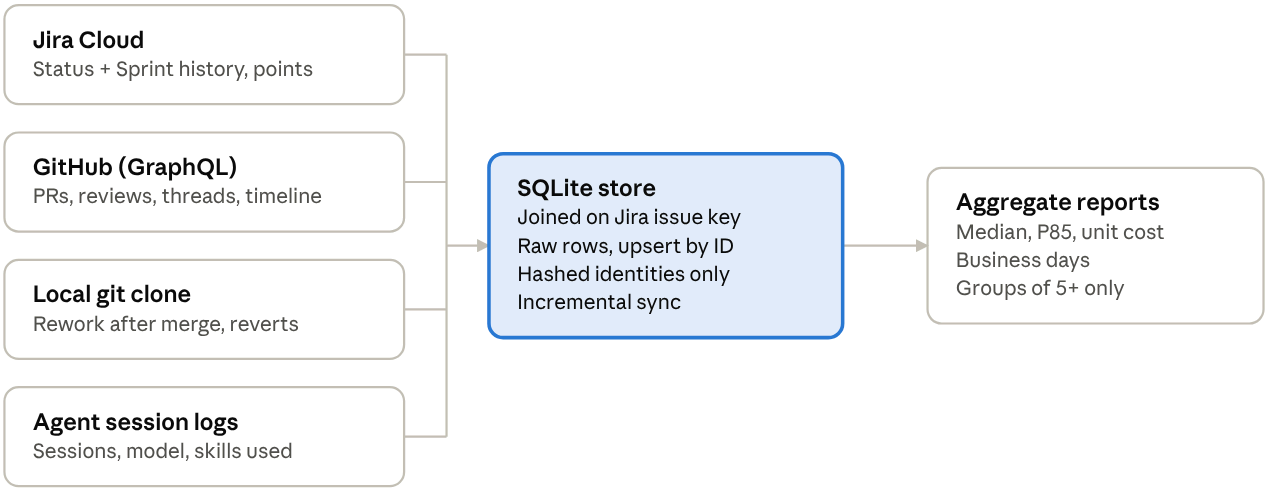
- **Against their own past.** We measure each participant's before-and-after change against everyone else's change over the same period. This cancels out differences in skill, seniority and codebase familiarity.
- **Group by what is known before work starts.** Story points (plus repo or epic) are used to group issues. Hours, PR size and cycle time are results, never groupings, because AI changes them directly.
- **Business days, not clock time.** Weekends and company holidays are excluded. Clock-time figures are reported alongside so our numbers can be compared with other teams'. Forecast dates are converted back to calendar dates.
- **Median and P85, not averages.** Cycle time data is skewed, so one stuck ticket distorts an average. Averages are used only for money that gets added up.
- **Privacy built in.**
  - Identities are pseudonymized with a salted hash when data is collected. Only hashes are stored.
  - Only aggregates are reported. Nothing is shown for groups under about 5 people.
  - Issue keys never appear next to results grouped by person or by skills group.
  - The tool cannot produce per-person reports.
  - Participants are told exactly what is collected before the pilot starts.

Token usage exists only on Pro accounts; the flat-rate accounts don't report it. Everywhere else, sessions and skills used per issue measure how much AI an issue got.

Data collection

All four sources sync into one SQLite store, joined on the Jira issue key that every branch name already carries.

Four sources sync into one store, joined on the Jira issue key



data pipeline · 4 sources, 1 store

Raw rows are stored and metrics are computed from them, so a definition can change without fetching everything again.

Source	What we pull	How
Jira Cloud	Status and Sprint field history, sprint dates, points, worklogs	`changelog/bulkfetch` filtered to status + Sprint; Agile API for sprints; sync on `updated >= last sync`, upsert by history ID
GitHub	PRs, reviews, review threads, timeline events, PR size	GraphQL, one query per PR
Local git clone	Rework within 3 weeks of merge, reverts	`git log` / `git blame`
Agent session logs	Sessions, model, skills used, branch (none = research); tokens on Pro accounts only	Import, linked to the issue through the branch name

Metrics

Three headline metrics carry the story, one per claim. Guardrails show where AI could make things worse.

Metric	Claim	Role	Source
Review rounds per PR (stand-in for senior review effort)	1. Review time	Headline	GitHub
"Not fixed" threads per PR	1. Review time	Supporting	GitHub review threads
Time to first review, time to re-review, share of PR life waiting on reviewer	1. Review time	Supporting	GitHub timeline
P85 cycle time by point bucket, and the gap between median and P85	2. Predictability	Headline	Jira
Time in each status per sprint	2. Predictability	Supporting	Jira
Delivery forecast range (SLE, Monte Carlo)	2. Predictability	Supporting	Jira
Merged PRs and closed issues per seat, by plan tier and point bucket	3. Seat value	Headline	Sessions + Jira + GitHub
Share of ticket-branch sessions with no merged result	3. Seat value	Supporting	Sessions + GitHub
Tokens and credits per merged PR (Pro accounts only)	3. Seat value	Supporting	Sessions
Reopen rate, revert rate, rework within 3 weeks, linked bugs	4. Quality	Guardrail	Jira + git
PR size; reviews per active reviewer	4. Quality	Guardrail	GitHub
Hours per point	—	Only if worklogs pass reliability checks	Jira worklogs

Metric	Claim	Role	Source
Reviewer concentration, distinct reviewers	—	Context, not claimed for AI	GitHub

All durations are in business days. Each AI practice (pre-review by an agent, grill-me or to-prd before coding, agent help addressing review comments) is flagged per PR from the session logs, so its effect can be compared directly. Sessions with no ticket branch (research, exploration, tooling) are reported as their own share of sessions and credits, never counted as waste.

The pilot proposal

We propose a 3–4 sprint pilot, on existing seats plus one more Pro seat, that produces the evidence leadership has asked for, under conditions where AI's value can actually show up.

- **Who:** the group already volunteering to use the mattpocock/skills workflow.
- **What changes:** nothing for the skills group. Structured vs. ad hoc use is measured on the $40 seats everyone already has. Tier evidence comes from the bullets below.
- **Added cost $60 a month**: one more Pro seat. Credit use on both Pro seats is tracked to size the $200 tier.
- **Second Pro seat:** goes to a mid-level developer in the skills group, with their consent, as a second named case study. Senior and mid-level results side by side show whether the tier gap holds where review pain is worst.
- **Replay benchmark:** rerun 10–20 past tickets from the commit before their merge, in four conditions: each model tier, with and without skills. Scored on the merged PR's own tests passing, human interventions needed, credits used, and a blind senior review of the diff. This separates the model's effect from the workflow's, using only the author's accounts.
- **$200 tier by demand, not ranking:** credit use on the two Pro seats shows whether heavy users outgrow the $100 tier. Credit exhaustion shows need without ranking people by output.
- **Early signal, starting now:** the author's own Pro accounts (single-user, tied to SSO) alternate issues between the newest models and Sonnet 4.6/Opus 4.5. That gives early data on tier differences and credit burn before trial seats arrive. Sessions are

split into delivery and research, since this account's credit use includes research such as this project. One senior engineer's results are a signal, not proof.

- **Success criteria written down before it starts**, taken from the headline and guardrail metrics and measured against the group's own baseline. They are sent to leadership as a dated record, so no one can say they were chosen after the fact.
- **A decision at the end:** expand, adjust or stop.

Break-even: the Pro upgrade costs $60 per person per month. Divided by a loaded engineer's monthly cost, that is the improvement needed to break even, typically under 1%.

The result, in one line: *"Structured AI use cut senior review rounds by N% and tightened our 85% delivery forecast from X to Y business days, for $60 more per person per month, with no increase in defects. Here is where it didn't help yet."*

Open questions:

- [ ] Which mid-level developer takes the second Pro seat?
- [ ] Do the Pro accounts' credits last a sprint of heavy use?
- [ ] Which past tickets go into the replay benchmark?
- [ ] Track this measurement work as its own Jira epic, so its AI use counts as attributable work?
- [ ] Which sprint starts the pilot, and how many sprints of baseline history do we have?
- [ ] When do the written success criteria go to leadership?

Presenting it

Lead with the problem leadership already feels, not with AI.

1. **The problem:** senior review is the bottleneck. PRs wait X business days for a first review and take X rounds to merge.
2. **One concrete example:** a real epic, start to finish, next to a similar epic from before as the baseline.
3. **The numbers:** the three headline metrics, with the guardrails holding.
4. **What didn't improve**, stated plainly.
5. **The ask:** the next step only, sized to the evidence.